

Typescript

mapping

Typescriptは他のタイプの属性をもとにして新しいタイプを生成することがある。

```
type Animals = "aligator" | "baboon" | "cat";

type AnimalCounts = {
  [K in Animals]: number;
};
```

mappingされたタイプはKeyセットのKeyについて新しい属性を作って新しいタイプを生成する。

```
interface AnimalVariants {
  alligator: boolean;
  baboon: number;
  cat: string;
}

type AnimalCounts = {
  [K in keyof AnimalVariants]: number;
};
```

一般的にMappingされたタイプは存在するタイプにkeyofを使ってkeyを持つ方式である。

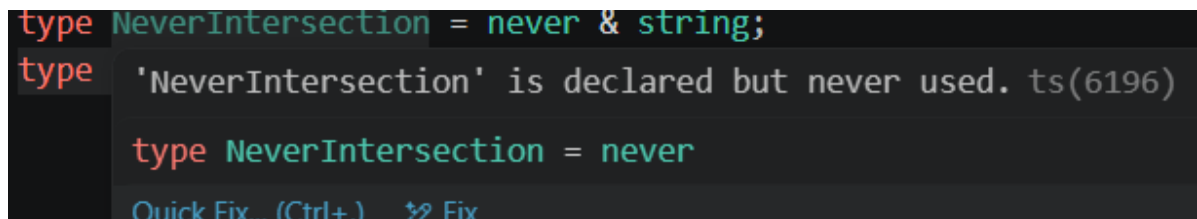
存在するタイプのkeyをmappingするようにすると、新しいタイプにmappingする。

制限

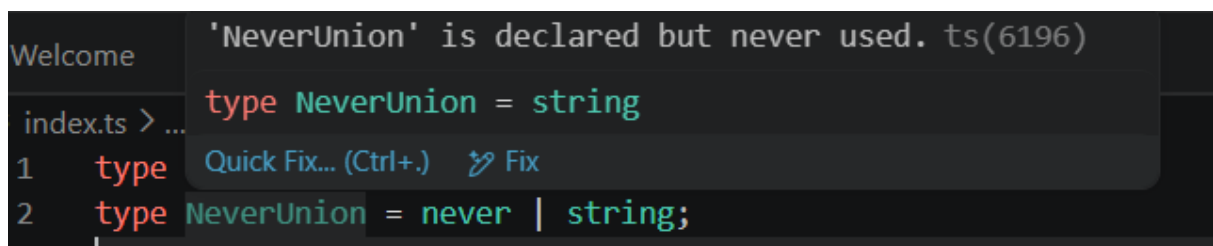
```
interface Environmentalist {  
  area: string;  
  name: string;  
}  
  
type ReadonlyEnvironmentalist = {  
  readonly [K in keyof Environmentalist]: Environmentalist[K];  
};  
  
type OptionalReadonlyEnvironmentalist = {  
  [K in keyof ReadonlyEnvironmentalist]?: ReadonlyEnvironmentalist[K];  
};
```

mappingされたタイプは元のタイプのアクセス修飾子Readonly、?も変更することができる。

never



```
type NeverIntersection = never & string;  
type 'NeverIntersection' is declared but never used. ts(6196)  
type NeverIntersection = never  
Quick Fix... (Ctrl+.)  Fix
```



```
Welcome  
index.ts > ...  
1 type  
2 type NeverUnion = never | string;  
Quick Fix... (Ctrl+.)  Fix
```

交差タイプ&にneverが含まれる場合、全体がneverになる。
共用体タイプ | にneverが含まれる場合、neverは無視される。